



FOR JAVA DEVELOPERS MOVING TO NESTJS

Test 2: two rows.

Spring Boot to NestJS, episode 18: integration tests with PostgreSQL

@DataJpaTest gives you rollback as part of the test slice. A Nest testing module gives you the application, not database isolation.

Two tests against a real Postgres, each inserting one row and counting: 1 and 1 under @DataJpaTest, 1 and 2 in Nest, and 3 and 4 on the next run.

The rollback comes from the test transaction. A plain @SpringBootTest also counted 1 and 2, and a test transaction stops at a real HTTP request on both stacks.

For this suite, truncating before each test was the reliable boundary, with care for foreign keys and shared databases.

In scope

- @DataJpaTest and @SpringBootTest against a Nest testing module
- MockMvc against supertest
- Where a test transaction stops
- Resetting data between tests

Not in scope

- Testcontainers: next episode
- Parallel test databases

One Postgres 18 from compose, test files run one at a time.

01 The measurements

SPRING @DataJpaTest	1, then 1
SPRING @SpringBootTest, NO @Transactional	1, then 2
NEST TESTING MODULE, TYPEORM	1, then 2; next run 3, then 4
NEST, QUERYRUNNER TRANSACTION PER TEST	the injected repository's row survived
@Transactional TEST, MockMvc	0 rows afterwards
@Transactional TEST, REAL HTTP	1 row afterwards
NEST, SUPERTTEST INSIDE A ROLLED-BACK TRANSACTION	1 row afterwards
NEST, TRUNCATE BEFORE EACH TEST	1, then 1

Two tests, one row each, then count.

02 Reset the data before each test

It works whichever connection did the write, for repository tests and requests alike. Truncate related tables together or cascade, and do not let parallel files share one database.

```
cleanup.spec.ts
```

```
beforeEach(async () => {  
  await dataSource.query('TRUNCATE ep17_notes_clean RESTART IDENTITY');  
});
```

two tests with TRUNCATE before each: 1 and 1

Run on a real machine, 2026-09-30, by scripts/verify-integration.sh in the course repository.

03 Where a test transaction stops

A QueryRunner transaction only covers writes made through runner.manager. The injected repository and a supertest request use their own connections.

```
cleanup.spec.ts
```

```
await notes.save({ text: 'injected' }); // kept  
await runner.manager.save(CleanNote, { text: 'runner' }); // rolled back
```

after both rolled-back tests, 1 row left: "injected"

Run on a real machine, 2026-09-30, by scripts/verify-integration.sh in the course repository.

Before you trust a NestJS integration suite

1. Decide how each test gets clean data
2. Truncate before each test for HTTP tests
3. Use a test transaction only when every write goes through its manager
4. Truncate related tables together, or cascade
5. Do not let parallel test files share one database
6. Build the testing module once per file where you can

Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- Every measurement, one command: scripts/verify-integration.sh**
<https://github.com/code-with-sam-dev/spring-to-node>
- NestJS testing**
<https://docs.nestjs.com/fundamentals/testing>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.